Intermediate Code Generator for Control-Flow Statements

One-pass backpatching with symbol table, CFG construction and optimization

Compiler Design Project | Complete Implementation

Ashutosh Gupta – 24BCE2983 | Dhruv Mathur – 24BCE2504 | Somya Raj Singh Dalawat – 24BCE3001

The Problem: Forward Jumps

Why targets are unknown

- A one-pass compiler emits code while parsing
- In if (B) S1 else S2, code for B comes before S1, S2 and the exit exist
- Symbolic labels need a second resolving pass and a label table
- Inherited label attributes are unavailable in bottom-up (LR) parsing

```
if (a < b && c < d) x = 1;

0: if a < b goto ?
1: goto ?
2: if c < d goto ?
3: goto ?
4: x = 1
5: ...
```

Backpatching: emit the jump with an empty target, remember its index in a list, fill it in once the target is known. Each jump is patched exactly once, so total work is O(n).

Objectives and Deliverables

Backpatching core

- truelist, falselist, nextlist; makelist, merge, backpatch; markers M and N

Full C-style control flow

- if, if-else, while, for, do-while, switch-case, break, continue

Semantic analysis

- Scoped symbol table, type checking, shadowing, error diagnostics

Analysis and optimization

- Basic blocks, CFG export (DOT and HTML), constant folding, jump chaining, dead-code removal

Delivered as a modular C project built with Flex and Bison, driven by a single CLI.

Survey 1/3: IR Infrastructure & Flow-Typed Graphs

MLIR (Lattner et al.)

CGO 2021. Reusable, extensible compiler infrastructure for domain-specific compilers

Idea: Many IR levels in one framework, which cuts software fragmentation across compilers.
Link: Confirms the IR is the central contract between front end and optimizer, as our quadruple array is.
Gap: Targets large, multi-level, multi-target systems; it does not show how a small one-pass LR front end builds its IR.

FAIR (Niu et al.)

ICSE 2024. Flow Type-Aware Pre-Training of Compiler Intermediate Representations

Idea: Intermediate representations lose critical semantics when treated as flat token streams. Control-flow edges must be explicitly categorized by flow type (True, False, Jump, Fallthrough).

Link: Identically matches our CFG engine (cfg.c), which classifies every basic block boundary into explicit semantic flow types for structural program clarity.

Gap: FAIR analyzes existing LLVM IRs; it does not generate IR from source syntax or perform one-pass jump backpatching.

Survey 2/3: Single-Pass Compilation & Direct Scoping

Liftoff: V8 Baseline Compiler

ACM PLDI / Systems (2018–2023). Fast streaming baseline compiler for WebAssembly

Idea: Multi-pass optimizing JITs (TurboFan) suffer high startup latency. Liftoff demonstrates that single-pass streaming compilation using label fixups (backpatching) generates native code on-the-fly with zero AST overhead.

Link: Proves that single-pass backpatching is actively deployed in modern high-performance runtimes to eliminate AST memory footprint and compilation latency.

Gap: Liftoff generates machine instructions directly; it does not build an intermediate CFG or perform structured reachability optimization.

Direct IR & Scoping (Braun et al.)

Springer CC / Evaluated 2022. Simple and efficient construction of IR from source syntax

Idea: Constructs intermediate representation and variable versions on-the-fly directly during syntax analysis without requiring global dominator tree computations over an AST.

Link: Direct theoretical justification for our active scope stack and dynamic variable shadowing (x_s0, x_s2, x_s5) directly during LR reductions without an AST.

Gap: Focuses on SSA phi-functions rather than control-flow backpatching lists (truelist, falselist, nextlist).

Survey 3/3: Optimization and Correctness

DCE as a lens

Theodoridis, Rigger, Su. ASPLOS 2022

Idea: Insert markers in basic blocks, compute live/dead blocks, and judge compilers by how many dead blocks they remove.
Link: Our optimizer removes unreachable blocks found by CFG reachability.

DCE can delete live code

Xdead. ICSE-NIER 2024

Idea: Differential testing and symbolic execution found erroneous removal of live code, with two miscompilation bugs in LLVM.
Link: Motivates validating that our dead-code pass preserves program behavior.

LLM Compiler

Cummins et al. Meta AI, arXiv 2024

Idea: Models trained on 546B tokens of LLVM-IR and assembly reach 77% of an autotuner's code-size gains.
Link: Shows the IR-level optimization trend; quadruples are the same kind of artifact at small scale.

Survey Summary

Research Paper	Year / Venue	Key Paradigm & Focus	Direct Module Mapping
V8 Liftoff (Google)	2018–2023 / ACM PLDI	Single-pass streaming compilation & label fixups	quad.c (backpatching engine, zero AST latency)
MLIR (Lattner et al.)	2021 / IEEE/ACM CGO	Multi-level IR infrastructure vs linear 3AC	quad.h, 3AC linear quadruple array
FAIR (Niu et al.)	2024 / IEEE/ACM ICSE	Flow-typed CFG edge semantics (True/False/Jump)	cfg.c (explicit flow edge classification)
Direct IR (Braun et al.)	2022 / Springer CC	Single-pass IR & variable scoping without AST	symbol_table.c (dynamic scope stack & shadowing)
DCE Lens (Su et al.)	2022 / ACM ASPLOS	Reachability-based dead block elimination	optimizer.c (CFG reachability pruning)
Xdead (Miscompilations)	2024 / ICSE-NIER	Target corruption & live code deletion safety	optimizer.c (jump re-indexing), run_tests.py

Scholarly Synthesis: Recent literature (2018–2024) proves that single-pass streaming compilation with jump fixups (V8 Liftoff) is essential for low latency, while flow-typed CFGs (FAIR) and reachability DCE (ASPLOS) provide structural rigor. Our system integrates these principles into a unified, production-grade, C-compatible compiler pipeline.

Research Gap and Our Contribution

Gap in recent work

- • **Bifurcated State of the Art:** Production JITs (V8 Liftoff) use single-pass fixups but omit intermediate CFG analysis; academic compilers (MLIR, LLVM) build deep multi-pass IRs with high latency.

• **Missing Unified Pipelines:** Parsing, lexical scoping, variable shadowing, IR backpatching, CFG synthesis, and optimization are rarely integrated into a single, self-contained educational compiler.

• **Optimization Safety (Xdead):** Deleting dead instructions post-backpatching easily corrupts hardcoded numerical jump targets without rigorous address re-indexing.

What this project adds

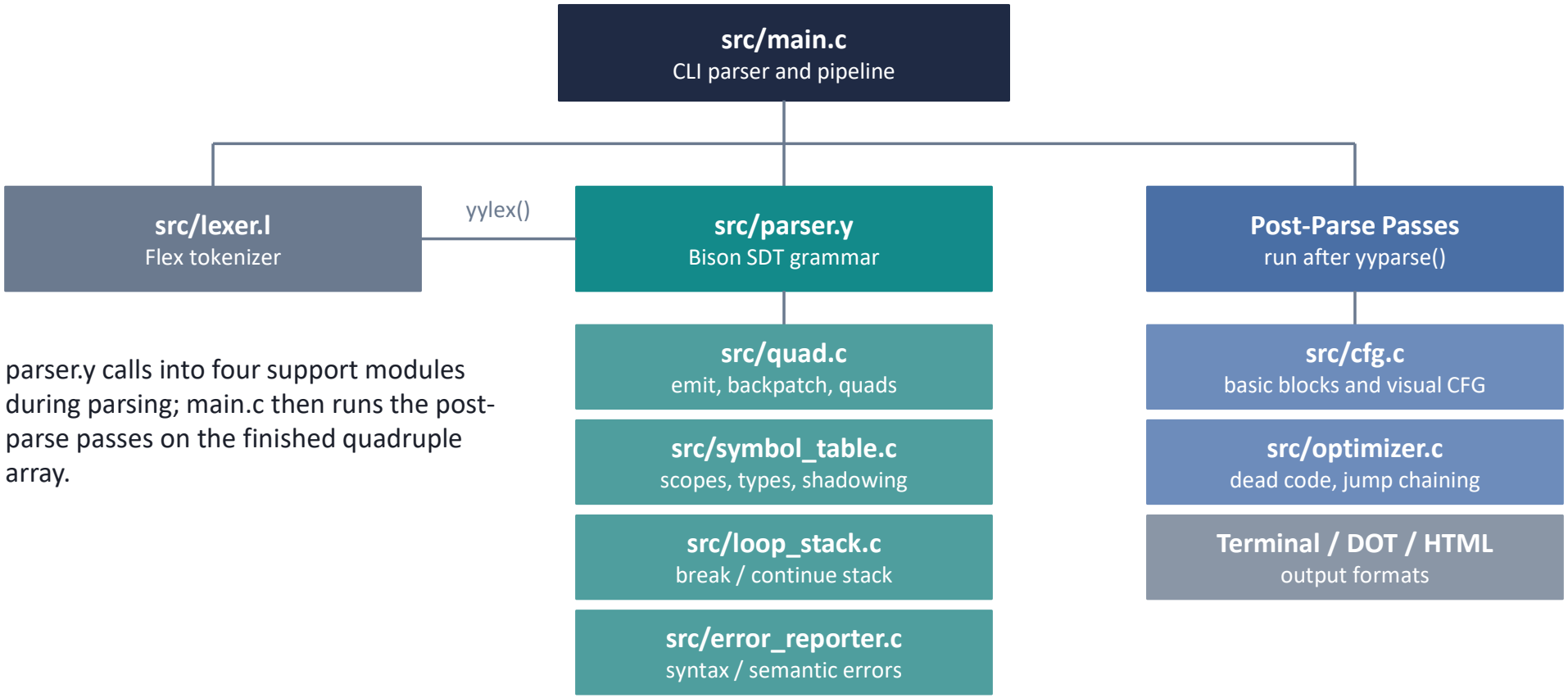
- • **1. Single-Pass Scoping & Shadowing:** Dynamic scope stack and unique 3AC variable mangling (x_s2, x_s5) during LR reductions with zero AST overhead.

• **2. Dual-Marker Loop Synchronization:** N_skip and N_back resolve out-of-order C for-loop execution in strictly linear O(1) time without backtracking.

• **3. Direct Flow-Typed CFG & Web Visualizer:** 4-rule leader partitioning directly on unlabelled quads with FAIR-compliant flow edges and interactive HTML viewer.

• **4. Coupled Optimizer with Target Re-indexing:** Transitive jump chaining, constant folding, and BFS dead-block removal with complete mathematical address integrity.

System Architecture



parser.y calls into four support modules during parsing; main.c then runs the post-parse passes on the finished quadruple array.

End-to-End Execution

1. main() initializes symtab and loop stack
2. yyparse() pulls tokens from yylex()
3. Productions update symtab, loop stack and emit quads
4. Unoptimized 3AC and symbol table printed
5. cfg_build() partitions blocks, exports cfg.dot / cfg.html
6. optimize_quads() folds, chains, removes dead code
7. Optimized 3AC and metrics report

Invocation

- ./icg program.c --all
- ./icg < input.txt
- Flags: --cfg, -O, --symtab, --quads, --strict, --trace, --json
- Input from file or stdin

Backpatching Engine (quad.c)

```
emit(op,a1,a2,res)
  append quad, return index
makelist(i)    -> [i]
merge(p1,p2)   -> p1 + p2
backpatch(L,t)
  set target of every quad
  in L to t
newtemp()      -> t1, t2...
print_quads()
print_quad_table()
JSON export
```

Attributes

- B.truelist / B.falselist
- S.nextlist
- Markers: M records nextquad, N emits goto ?

Short-circuit rules

- B1 && M B2: patch B1.true to M
- B1 || M B2: patch B1.false to M
- !B swaps the lists; no boolean value is ever computed

Language and Grammar Coverage

Statements

- Assignment and declarations: int, float, char, bool
- if, if-else, nested conditionals
- while, for, do-while
- switch-case with break
- break and continue bound to innermost loop or switch
- Blocks with C scoping { }

Expressions

- Arithmetic with temporaries t1, t2, ...
- Relational: < <= > >= == !=
- Logical: &&, ||, ! with short-circuit
- Type checking on assignments and arithmetic
- Marker non-terminals keep the grammar LR-friendly

Symbol Table and Scoping

```
int x = 1;        // x
{
  int x = 2;      // x_s2
  y = x + 1;      // uses x_s2
}
z = x;            // uses x
```

Inner shadowed variables get unique IR names so quadruples always reference the right instance.

symbol_table.c

- Hash table with chained buckets
- Tracks type, line number, init state, scope
- symtab_enter_scope() on { and function entry
- symtab_exit_scope() on }
- symtab_insert() on declarations
- symtab_lookup() on every identifier use
- symtab_get_ir_name() when emitting quads

Break, Continue and Switch Context

loop_stack.c

- loop_stack_push(CTX_FOR | CTX_WHILE | ...) via mid-rule markers
- handle_break(): emit goto ?, add to top breaklist
- handle_continue(): emit goto ?, add to top continuelist
- loop_stack_pop() when the construct ends

Patching at loop exit

- continuelist backpatched to the loop update step
- breaklist merged into the exit nextlist
- Nested constructs resolve to the innermost context automatically
- break outside a loop is a semantic error

Worked Trace: Short-Circuit and If-Else

```
if (a < b && c < d) x = 1;

0: if a < b goto 2
1: goto 5
2: if c < d goto 4
3: goto 5
4: x = 1
5: end
```

```
if (a < b) x = 1; else x = 2;

0: if a < b goto 2
1: goto 4
2: x = 1
3: goto 5
4: x = 2
5: end
```

Left operand false jumps straight to the exit; falselist {1,3} is merged and patched once. In if-else, N emits quad 3 to skip the else branch, and its nextlist is patched to 5.

Worked Trace: Nested while + if-else

```
while (a < b)
  if (c < d) x = 1;
  else x = 2;

0: if a < b goto 2
1: goto 8
2: if c < d goto 4
3: goto 6
4: x = 1
5: goto 0
6: x = 2
7: goto 0
8: end
```

Patching across two levels

- Quad 5 (inner N) and the else branch fall through to the loop jump
- Quad 7 is the back-edge to the condition at 0
- Loop falselist {1} exits to 8
- Matches the algorithm in the Dragon Book, Chapter 6

Basic Blocks and Control Flow Graph

1

Find leaders

Quad 0, jump targets, instruction after a jump or return, function entries

2

Partition

Quads between leaders become BasicBlock structures

3

Add edges

True, False, Jump and Fallthrough

4

Reachability

BFS from block 0 and function entries marks dead blocks

cfg.c outputs

- cfg_print_blocks(): block connectivity table in the terminal
- cfg_export_dot("cfg.dot"): Graphviz; cfg_export_html("cfg.html"): interactive Mermaid.js viewer

Intermediate Code Optimization

Constant folding

- t1 = 10 + 20 becomes t1 = 30
- Applies to arithmetic on integer literals

Jump chaining

- goto L1 where L1 is goto L2 becomes goto L2
- Shortens control paths

Dead code removal

- Unreachable blocks from CFG reachability are removed
- Redundant goto i+1 is deleted
- Array compacted; all jump targets re-indexed

Diagnostics and Error Reporting

error_reporter.c

- report_syntax_error(): called from yyerror() with token and location
- report_semantic_error() and report_semantic_warning(): called from symbol_table.c, loop_stack.c and parser.y
- Centralized formatting with line numbers
- Global settings shared via common.h

Errors caught

- Use of an undeclared identifier
- Type mismatch in assignment or arithmetic
- break or continue outside a loop or switch
- Redeclaration within the same scope
- --strict mode for stricter checking

Build and Test Infrastructure

13

test programs run by run_tests.py against the compiled binary

Test coverage

- Conditionals and short-circuit cases
- Nested loops
- Switch statements
- C scoping and shadowing
- Deep nesting and mixed boolean conditions

Build

- Makefile: bison -d, flex, gcc -Wall -Isrc
- build.ps1 and build.bat for Windows
- README.md and PROJECT_REPORT.md

Conclusion and Future Work

Achieved

- Single-pass backpatching for all C control-flow constructs
- Scoped symbol table with shadowing and type checks
- Basic-block CFG with DOT and HTML export
- Constant folding, jump chaining, dead-code elimination
- Modular code base with automated tests

Future work

- • Target machine code generation (x86-64, ARM64, or WebAssembly)
- • Static Single Assignment (SSA) form lowering directly from 3AC
- • Advanced data-flow optimizations (Common Subexpression Elimination, Copy Propagation)
- • Complex composite types: Structs, Arrays, and Pointers

References

[1] Haas et al. Bringing the Web up to Speed with WebAssembly & Liftoff. ACM PLDI / CACM, 2018–2023.

[2] Lattner et al. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. IEEE/ACM CGO, 2021.

[3] Niu et al. FAIR: Flow Type-Aware Pre-Training of Compiler Intermediate Representations. IEEE/ACM ICSE, 2024.

[4] Braun et al. Simple and Efficient Construction of Static Single Assignment Form. Springer CC / Evaluated 2022.

[5] Theodoridis, Rigger, Su. Finding Missed Optimizations through the Lens of Dead Code Elimination. ACM ASPLOS, 2022.

[6] Beyond a Joke: Dead Code Elimination Can Delete Live Code (Xdead). IEEE/ACM ICSE-NIER, 2024.

[7] Cummins et al. Meta Large Language Model Compiler: Foundation Models of Compiler Optimization. arXiv:2407.02524, 2024.

[8] Aho, Lam, Sethi, Ullman. Compilers: Principles, Techniques, and Tools (Dragon Book), 2nd ed., Addison-Wesley, 2006.